TIPE 2024 – 2025 :

Le chiffrement et déchiffrement de messages

Thème :
Transition, transformation, conversion

Samuel PLACEK

MP Option Informatique

Plan

- Introduction
- Code de César
- Chiffrement RSA
- Conclusion

Introduction

Cryptographie : pilier de la sécurité des communications numériques modernes ⇒ Mails, transactions bancaires, messages

Cache des défis mathématiques / informatiques : Arithmétique modulaire, modélisation par programmes informatiques complexes.

Cryptographie asymétrique : Clé privée et publique ⇒ assure plus de sécurité de chiffrement d'un message

Objectifs :

i) Comprendre l'évolution des systèmes de cryptographie

ii) Trouver des failles pour décrypter un message sans clé privé

⇒ Mise en place d'une méthode d'attaque / Implémenter, vérifier l'efficacité

I – Code de César

Contexte : Créé / utilisé par Jules César

But : Communiquer avec ses troupes.
Connu pour sa simplicité pour chiffrer / déchiffrer un message.

Principe : Translate l'alphabet :
Ex : a -> c, b -> d, …, y -> a, z -> b : décalage de 2 lettres

Ici : Alphabet français : a, b, …, z sans caractères spéciaux ni majuscules.
26 décalages possibles (0 : mot inchangé)

Risque : nombre de décalages limité, faible = sécurité non assurée.

Deux Méthodes de Décryptage Possibles

1) Méthode ‘intelligente’

2) Méthode ‘brutale’

1) Méthode intelligente

Lettre la plus utilisée = lettre la plus présente dans le mot

Langue française : lettre ‘e’

Ex : Décrypter le mot ‘gpxkg’.

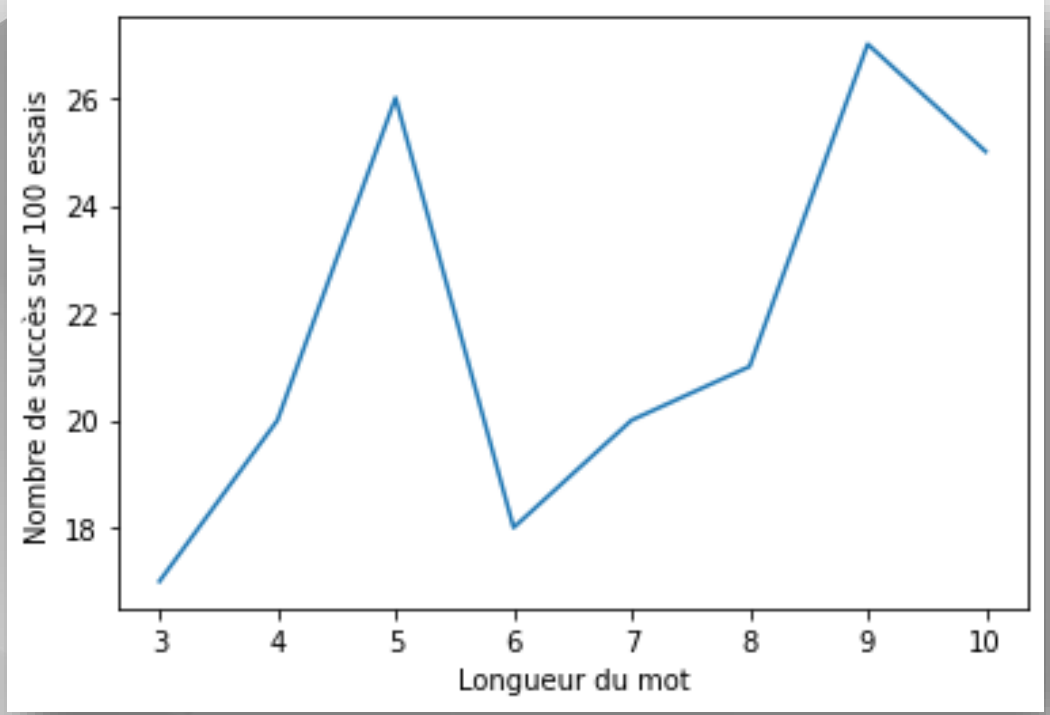
Lettre la plus présente : g.

Donc e → g ce qui donne un décalage de 2
Mot décrypté : ‘envie’ ⇒ cohérent.

Toutefois, pas très efficace

Ex : ‘vivre’ par exemple, 2 ‘v’.

Programme qui donne le nombre du succès pour des mots de longueur donnée



On observe :

i) Taux de réussite $< 30\% \Rightarrow$ pas fiable pour décrypter des phrases entières

ii) Dépend beaucoup du nombre de lettres dans le mot

Conclusion : Pas très pertinent.

2) Méthode brutale

26 décalages possibles → tester toutes les combinaisons et trouver le mot qui a du sens

Permet assurément de trouver le mot en temps raisonnable : $O(n), n = len(mot)$

salvadorien	nichan	total	rond	paros	raffoliez	fondant	raya	vives	soumettrait
dlwglozctpy	avpuna	mhmte	olka	itkhl	ajooxurni	oxwmjwc	nwuw	bobky	miogynnlucn
ckvfknybsox	zuotmz	lglsd	nkjz	hsjgk	zinnwtqmh	nwvlivb	mvtv	anajx	lhnfxmmktbm
bjuejmxarnw	ytnsly	kfkrc	mjiy	grifj	yhmmvspl	mvukhua	lusu	zmziw	kgmewlljsal
aitdilwzqmv	xsmrkx	jejqb	lihx	fqhei	xgllurokf	lutjgtz	ktrt	ylyhv	jfldvkkirzk
zhschkvyplu	wrlqjw	idipa	khgw	epgdh	wfkktqnje	ktsifsy	jsqs	xkxgu	iekcujjhqyj
ygrbgjuxokt	vqkpiv	hchoz	jgfv	dofcg	vejjspmid	jsrherx	irpr	wjwft	hdjbtiigpxi
xfqafitwnjs	upjohu	gbgny	ifeu	cnebf	udiirolhc	irqgdqw	hqoq	vives	gciashhfowh
wepzehsvmir	toingt	fafmx	hedt	bmdae	tchhqnkgb	hqpfcpv	gpnp	uhudr	fbhzrggenvg
vdoydgrulhq	snhmfs	ezelw	gdcs	alczd	sbggpmjfa	gpoebou	fomo	tgtcq	eagyqffdmuf
ucnxcfqtkgp	rmgler	dydkv	fcbr	zkbyc	raffoliez	fondant	enln	sfsbp	dzfxpeeclte
tbmwbepsjfo	qlfkdq	cxcju	ebaq	yjaxb	qzeenkhdy	enmczms	dmkm	rerao	cyewoddbksd
salvadorien	pkejcp	bwbit	dazp	xizwa	pyddmjgcx	dmlbylr	cljl	qdqzn	bxdvnccajrc
rzkuzcnqhdm	ojdibo	avahs	czyo	whyvz	oxcclifbw	clkaxkq	bkik	pcpym	awcumbbziqb
qyjtybmpgcl	nichan	zuzgr	byxn	vgxuy	nwbbkhea	bkjzwjp	ajhj	oboxl	zvbtlaayhpa
pxisxalofbk	mhbgzm	ytyfq	axwm	ufwtx	mvaajgdzu	ajiyvio	zigi	nanwk	yuaskzzxgoz
owhrwzkneaj	lgafyl	xsxep	zwvl	tevsw	luzzifcyt	zihxuhn	yhfh	mzmvj	xtzrjyywfny
nvgqvyjmdzi	kfzexk	wrwdo	yvuk	sdurv	ktyyhebxs	yhgwtgm	xgeg	lylui	wsyqixxvemx
mufpuxilcyh	jeydwj	vqvcn	xutj	rctqu	jsxxgdawr	xgfvsfl	wfdf	kxkth	vrxphwwudlw
lteotwhkbxg	idxcvi	upubm	wtsi	qbspt	irwwfczvq	wfeurek	vece	jwjsg	uqwogvvtckv
ksdnsvgjawf	hcwbuh	total	vsrh	paros	hqvvebyup	vedtqdj	udbd	ivirf	tpvnfuusbju
jrcmrufizve	gbvatg	snszk	urqg	ozqnr	gpuudaxto	udcspci	tcac	huhqe	soumettrait
iqblqtehyud	fauzsf	rmryj	tqpf	nypmq	fottczwsn	tcbrobh	sbzb	gtgpd	rntldssqzhs
hpakpsdgxtc	eztyre	qlqxi	spoe	mxolp	enssbyvrm	sbaqnag	raya	fsfoc	qmskcrrpygr
gozjorcfwsb	dysxqd	pkpwh	rond	lwnko	dmrraxuql	razpmzf	qzxz	erenb	plrjbqqoxfq
fnyinqbevra	cxrwpc	ojovg	qnmc	kvmjn	clqqzwtpk	qzyolye	pywy	dqdma	okqiappnwep
emxhmpaduqz	bwqvob	ninuf	pmlb	julim	bkppyvsoj	pyxnkxd	oxvx	cpclz	njphzoomvdo

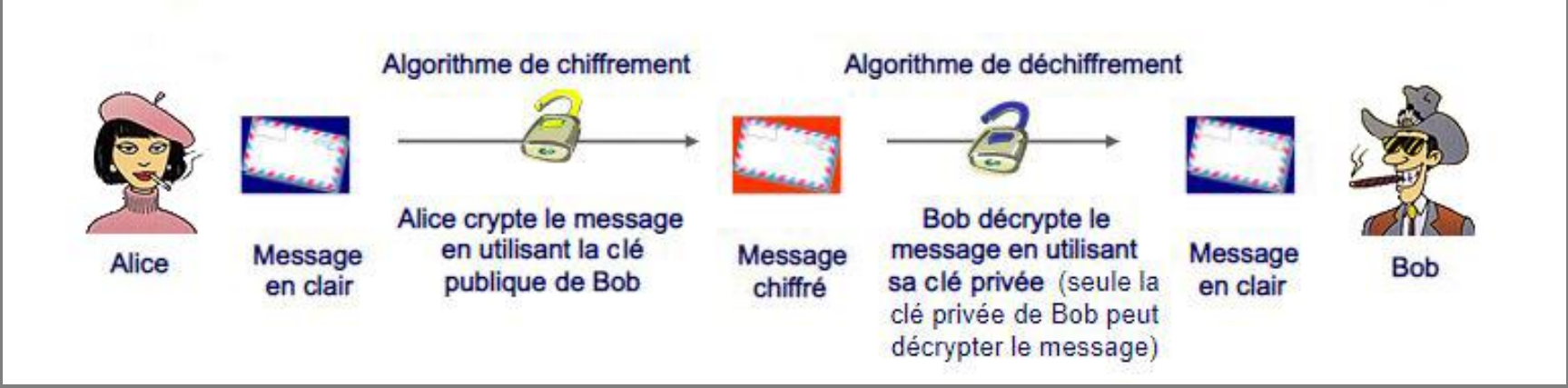
Décryptage de 10 mots aléatoire par analyse de toutes les possibilités

II – Chiffrement RSA

Contexte : Cryptographie asymétrique (clé privée et clé publique)

- **Très utilisé dans le commerce électronique / échanger des données sur Internet.**

Cryptographie asymétrique :



https://www.apprendre-en-ligne.net/crypto/rsa/index.html

A – Chiffrement

Définition : Soit $(p, q) \in \mathcal{P}, p \neq q$, on note $n = pq$ et $\varphi(n) = (p-1)(q-1)$

Soit e tel que e $< \varphi(n)$ et e $\wedge \varphi(n) = 1$

Lettre de rang $x \xrightarrow{\text{RSA}} y$ où $y \equiv x^e [n]$

Exemple : $p = 3, q = 11 \Rightarrow \begin{cases} n = 33 \\ \varphi(n) = 20 = 2^2 \times 5 \end{cases}$

On prend $e = 3$ ($e \wedge \varphi(n) = 1$)

Lettre $'f'$: $x = 5$; $x^e = 125 \equiv 26[33]$

B – Déchiffrement

Propriété : Soit y un entier quelconque.

Alors y code la lettre de rang x tel que $x \equiv y^d[n]$ où d est l'inverse modulo $\varphi(n)$ de e (i.e. $ed \equiv 1[\varphi(n)]$).

Vocabulaire : (n, e) : clé publique et (p, q) : clé privée

Exemple : $n = 33$ $(\varphi(n) = 20)$, $d = 7$ $(d \times e = 21 \equiv 1[20])$

Pour $y = 26$: $y^d = 8031810176 \equiv 5[33]$

Lettre de rang 5 : $'f'$ (rang $'a' = 0$)

C – Décryptage : Factorisation de n avec $n = pq$

Intérêt : Déterminer $\varphi(n)$ ⇒ connaitre le message

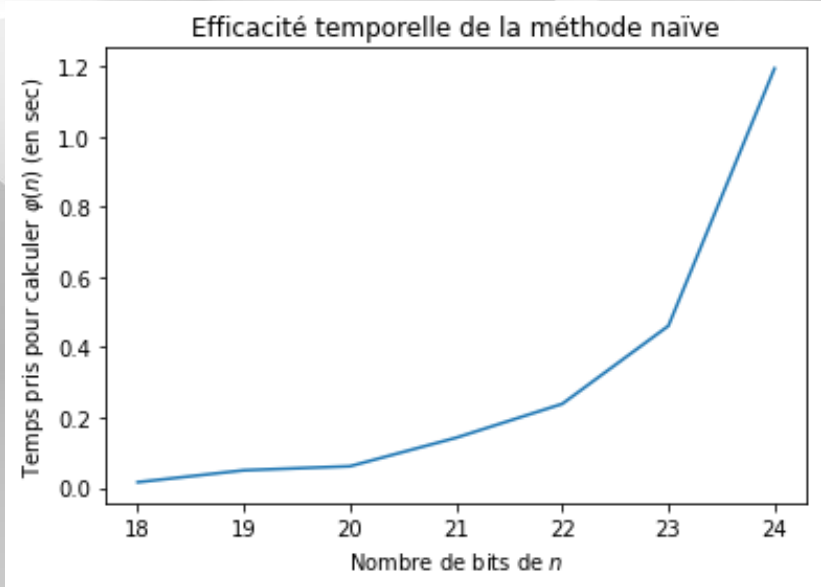
1 - Méthode naïve :

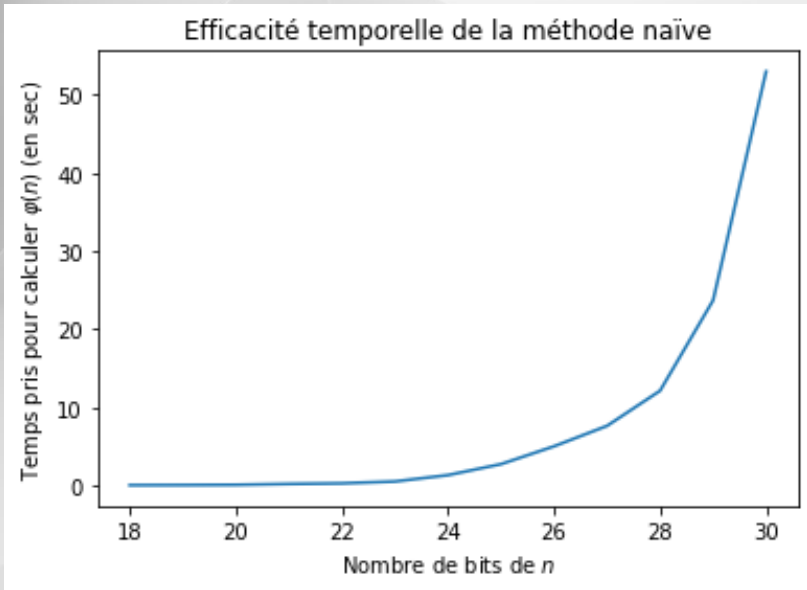
- Exhiber $\mathcal{D} = \{p \in P\ ; p|n\ et\ p \leq \sqrt{n}\}$
- Calculer $\varphi(n) = n\ \prod_{p|n}(1 - \frac{1}{p})$

✓ Assure le décryptage du message

✗ Que pour des n “petits” (cf. slide suivante)

Mise en évidence graphique de la méthode naïve





Ordinateur :

- CPU : 12th Gen Intel(R) Core(TM) i7-12700H 2.30 GHz
- RAM : 16,0 GB
- OS : Windows 11
- Python : 3.9.12

Croissance très rapide pour n > 26 bits !!

RSA : 1024 bits avant → 3000 de nos jours !

Conclusion : Utile pour $n \approx 10 - 40$ bits, très long sinon

2 - Algorithme ρ de Pollard

Principe : i) Génère une suite d'entiers mod n à partir d'une fonction f

Choix de f :

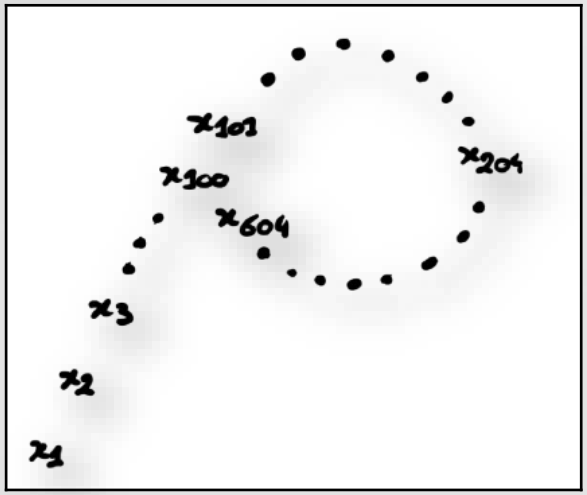
- Non linéaire : éviter cycles trop simples
- Facile à calculer
- Répartir les valeurs de manière imprévisible : augmenter les chances que $x_i \equiv x_j[p]$ où $p|n$

Ici $f : x \to (x^2+1)[n]$

- Simple : Facile à calculer, peu coûteuse en temps
- Non-linéaire : Contrairement à $x+1$, pas de séquences triviales
- Bonne dispersion : « $+1$ » : casse les symétries, collisions plus probables

Ainsi $(x_n)_{n\in\mathbb{N}}$ telle que $\begin{cases} x_0 = 2 \ (puisque \ on \ veut \ p \in \mathcal{P} \ soit \ p \geq 2 \\ \forall n \in \mathbb{N}, x_{n+1} = f(x_n) \end{cases}$

ii) Repérer un cycle → lettre ρ

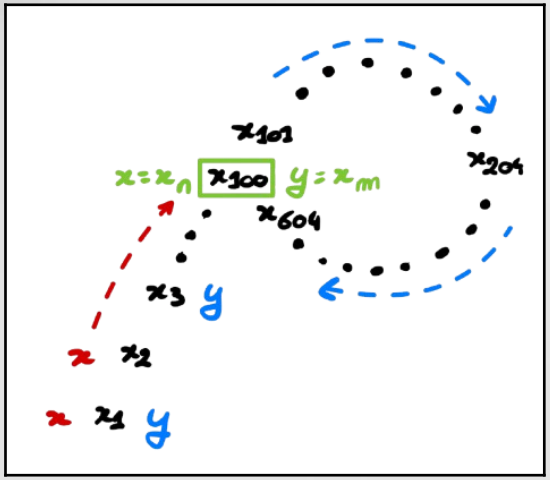


Exemple de représentation possible de la suite $(x_n)_{n\in\mathbb{N}}$

iii) On initialise x (la tortue) : avance de 1 pas et y (le lièvre) avance de 2 pas i.e. à chaque itération : $\begin{cases} x \to f(x) \\ y \to f(f(y)) \end{cases}$

iv) On travaille dans $\mathbb{Z}/p\mathbb{Z}$ où $p|n$ (cf. figure)

On repère lorsque x et y se croisent (arrive forcément car $\mathbb{Z}/p\mathbb{Z}$ est fini)



Evolution de x et y dans le graphe

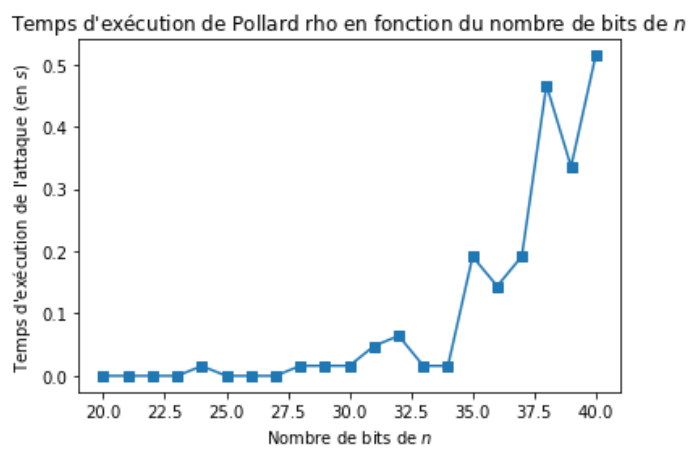
v) À ce moment, $x \equiv y[p]$ donc $x - y \equiv 0[p]$ i.e. $p|\ (x - y)$

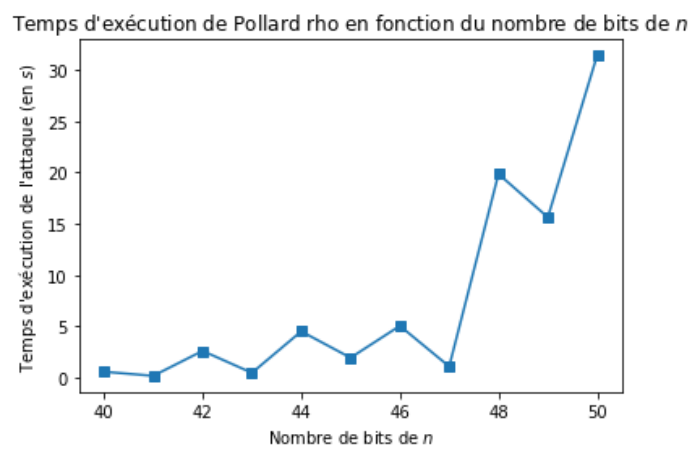
Ainsi, $p|\ pgcd(|x - y|, n) \coloneqq r$

- $r = 1$: continuer dans le cycle
- $1 < r < n$: r facteur non trivial de n
- $r = n$: recommencer avec un autre cycle

Implémentation informatique et résultats

Temps d'exécution





+ rapide que méthode d'avant

Temps raisonnable au delà de 50 bits

MAIS
7 min pour 60 bits → très long
RSA ≈ 3000 bits ⇒ très loin du vrai RSA

Taux de réussite



Chaque bit : 10 essais

Chaque essai : mot choisis aléatoirement (alphabet français)

Mot : longueur aléatoire (entre 1 et 26) jusqu'à obtenir un mot

✓ Bon taux de réussite jusqu'à 50 bits

✗ Au dessus de 50/60 bits, très mauvais taux

Conclusion

Pourquoi RSA ?

i) Présent dans la majorité des protocoles de sécurité (HTTPS)

ii) Permet le chiffrement sans échange de clé secrète (asymétrie)

iii) Robuste contre les attaques classiques

Mais limite de RSA :

i) Reste lent pour de grandes quantités de données

ii) Nécessité de clé très longues ($\approx$ 3000 bits)

Puissance de calcul des ordinateurs : limitée ⇒ impossibilité de décrypter des messages de nos jours

Mais : Ordinateurs quantiques ? qubits (bits qui peut être une « combinaison linéaire de 0 et 1 »)

Algorithme de Shor : factoriser n très rapidement : casser le RSA en temps polynomial en $\log(n)$ au lieu de exponentiel.

⇒ Décrypter un message sur 3000 bits quasi instantanément

FIN

Samuel PLACEK
MP Option Informatique

Annexes

Démonstration

On a : $ed \equiv 1[\varphi(n)] \Rightarrow k \in \mathbb{Z}, ed = k\varphi(n) + 1.$

Notons $r \coloneqq x \wedge n$. $r|n$ et $n = pq \Rightarrow r \in \{1, p, q, pq\}$

1e cas : $r = 1$

$p \in \mathcal{P}$ et $p \nmid x$ (sinon $r \neq 1$) $\Rightarrow p \nmid x^{q-1}$

Th de Fermat : $(x^{(q-1)})^{p-1} \equiv 1[p]$ i.e. $x^{\varphi(n)} \equiv 1[p]$

De même, $x^{\varphi(n)} \equiv 1[q]$

Donc $p\left|\left(x^{\varphi(n)} - 1\right),\ q\right|\left(x^{\varphi(n)} - 1\right)$

De plus, $p \wedge q = 1 \Rightarrow p \vee q = pq$

$\Rightarrow pq|\left(x^{\varphi(n)} - 1\right)$ d'où $x^{\varphi(n)} \equiv 1[n]$

Alors $x^{ed} = x^{1+k\varphi(n)} = x(x^{\varphi(n)})^k \equiv x[n]$

2ème cas : $r = p$ (ou $r = q$)

$q \in \mathcal{P}$ et $q \nmid x$ (sinon $r \neq p$) $\Rightarrow x^{(q-1)} \equiv 1[q]$ (Fermat)

Alors $x^{ed} = x^{1+k(p-1)(q-1)} = x(x^{q-1})^{k(p-1)} \equiv x[q]$

$\Rightarrow q|(x^{ed} - x)$.

De plus, $r = p|x \Rightarrow p|\ (x^{ed} - x)$

Donc $pq|\ (x^{ed} - x)$ i.e. $x^{ed} \equiv x[n]$

3ème cas : $r = pq$

$x \equiv 0[n] \Rightarrow x^{ed} \equiv 0[n]$

D'où $x^{ed} \equiv x[n]$

Conclusion :

Dans tous les cas $x^{ed} \equiv x[n]$

Or $y \equiv x^e[n] \Rightarrow y^d \equiv x^{ed}[n]$

Donc $x \equiv y^d[n]$

I - Génération d'un mot aléatoire (pour application César et RSA)

```python
import pandas as pd
import random
import re

def get_file():
    # Charger le fichier XLSB une seule fois
    fichier_xlsb = 'Lexique383.xlsb'
    df = pd.read_excel(fichier_xlsb, engine='pyxlsb')

    # Sauvegarder le DataFrame en Pickle
    df.to_pickle('Lexique383.pkl')

def generate_random_word(N):
    # Charger le fichier Pickle
    df = pd.read_pickle('Lexique383.pkl')
    # Filtrer les mots qui ont exactement la longueur choisie
    df_filtree = df[(df['15_nblettres'] == N) & (df['1_ortho'].notna())]

    # Transformer les mots filtrés en une liste et retirer les caractères spéciaux
    mots = df_filtree['1_ortho'].astype(str).tolist()
    mots_sans_accents = [mot for mot in mots if re.fullmatch(r'[a-z]+', mot)]

    # Choisir un mot aléatoire
    if mots_sans_accents:
        mot_aleatoire = random.choice(mots_sans_accents)
        return mot_aleatoire
    else:
        return 'None'
```

II – Code de César

1) Chiffrage et déchiffrage

```
# a = 97, z = 122 en ASCII

def codageCesar(t, d): # d >= 0
    t_bis = t[:]
    for i in range(len(t)):
        if ord(t[i]) + d <= 122:
            t_bis[i] = chr(ord(t[i]) + d)
        else:
            t_bis[i] = chr(97 + (ord(t[i]) + d) % 122-1)
    return t_bis


def decodageCesar(t, d):
    t_bis = t[:]
    for i in range(len(t)):
        if ord(t[i]) - d >= 97:
            t_bis[i] = chr(ord(t[i]) - d)
        else:
            t_bis[i] = chr(122 - (97 - (ord(t[i]) - d))+1)
    return t_bis
```

2) Méthode intelligente

Fonctions utiles à la méthode

```
def frequences(t):
    return {chr(i):t.count(chr(i)) for i in
range(97,123)}
def decodageAuto(t):
    occ = frequences(t)
    max_occ = max(occ.values())
    for k in occ:
        if occ[k] == max_occ:
            d = abs(ord('e') - ord(k))
            return decodageCesar(t, d)
```

Tracé de l'efficacité de la méthode

```
def mth_intel(N_min, N_max, m):
    # Determiner le nombre de reussite de la fonction decodageAuto sur m essais
    # pour un mot de longueur comprise entre N_min et N_max
    length = [N for N in range(N_min, N_max+1)]
    sucess = []
    for N in range(N_min, N_max+1):
        count = 0
        for i in range(m):
            mot = mot_random.generate_random_word(N)
            if mot != 'None':
                lst_mot = [char for char in mot]
                d = np.random.randint(0, 25)
                mot_chiffre = codageCesar(lst_mot, d)
                mot_decrypte = decodageAuto(mot_chiffre)
                if mot_decrypte == lst_mot:
                    count += 1
        sucess.append(count)

    # Représentation graphique
    plt.plot(length, sucess)
    plt.xlabel("Longueur du mot")
    plt.ylabel("Nombre de succès sur {} essais".format(m))
    plt.title("")
    plt.show()
    plt.savefig("mth_intel.png")
```

3) Méthode brute

```
def mth_brut(m):
    colonnes = {}
    for i in range(m):

        # Génère un mot de longueur aléatoire entre 2 et 12
        N = np.random.randint(2, 12)
        mot =  mot_random.generate_random_word(N)

        lst_mot = [char for char in mot]
        d = np.random.randint(0, 25)
        mot_chiffre = codageCesar(lst_mot, d)

        # Déchiffre avec tous les décalages possibles
        decryptages_possibles = []
        for p in range(26):
            decryptages_possibles.append("".join(decodageCesar(mot_chiffre, p)))

        if mot not in colonnes.keys():
            colonnes[mot] = decryptages_possibles

    # Transformer le dictionnaire en DataFrame
    df = pd.DataFrame(colonnes)

    # Exporter vers un fichier Excel
    df.to_excel('mth_brut.xlsx', index=False)
```

III – RSA

1) Chiffrage et déchiffrage

```
from random import randint
from sympy import randprime, gcd, mod_inverse
import matplotlib.pyplot as plt
import time
import mot_random

def encrypt(char_list, e, n):
    return [(pow(ord(c), e) - 1) % n  for c in
char_list]
def decrypt(cypher_list, d, n):
    return [chr(97 + (nb**d)%n) for nb in cypher_list]
```

2) Méthode naïve

Implémentation

```
def facteurs(n):
    facteurs = []
    for p in range(2,int(n**0.5)+1):
        if n % p == 0:
            facteurs.append(p)
            while n % p == 0:
                n //= p
    if n > 1:
        facteurs.append(n)
    return facteurs

def phi(n):
    res = n
    fact = facteurs(n)
    for p in fact:
        res *= (1- 1/p)
    return int(res)


def mth_naive(t, n, e):
    d = mod_inverse(e, phi(n))
    msg = []
    for nb in t:
        msg.append(chr(97 + (nb**d)%n))
    return msg
```

Tracé du temps de calcul

```
def tps_calcul(n):
    t1 = time.time()
    p = phi(n)
    t2 = time.time()
    return t2-t1

def trace():
    bits_list = list(range(18, 21))
    N = [generate_rsa_keys(bits)[0][1] for bits in bits_list]
    T = [tps_calcul(n) for n in N]
    plt.plot(bits_list, T)
    plt.xlabel("Nombre de bits de $n$")
    plt.ylabel("Temps pris pour calculer " + r"$\varphi(n)$" + " (en sec)")
    plt.title("Efficacité temporelle de la méthode naïve")
    plt.show()
```

3) Algorithme de Pollard ρ

```python
# Récupération d'un facteur de n par algorithme de pollard rho
def pollard_rho(n):
    if n % 2 == 0:
        return 2  # cas rapide si N est pair
    def f(x):
        return (x * x + 1) % n

    x, y, d = 2, 2, 1
    while d == 1:
        x = f(x)
        y = f(f(y))
        d = gcd(abs(x - y), n)
    if d == n:
        return None
    return d


# Simulation d'une attaque par pollard rho
def attaque_pollard(cipher_list, e, n):
    p = pollard_rho(n)
    if not p:
        return None
    q = n//p
    phi = (p-1)*(q-1)
    try:
        d = mod_inverse(e, phi)
    except ValueError:
        return None
    try:
        decrypted = decrypt(cipher_list, d, n)
    except:
        return None
    return decrypted


def est_decrypte(msg, bits):
    public, _, _ = generate_rsa_keys(bits)
    e, n = public
    cipher = encrypt(msg, e, n)
    return msg == attaque_pollard(cipher, e, n)
```

Tracé du temps d'exécution

```
def temps_pollard(bits):
    public, _, _ = generate_rsa_keys(bits)
    e, n = public

    msg_length = randint(1, 26)
    msg = list(mot_random.generate_random_word(msg_length))
    cipher = encrypt(msg, e, n)

    t1 = time.time()
    res = attaque_pollard(cipher, e, n)
    t2 = time.time()
    return t2 - t1

def trace_time(min_bits, max_bits):
    times = []
    sizes = []
    for bits in range(min_bits, max_bits + 1):
        sizes.append(bits)
        times.append(temps_pollard(bits))
    plt.plot(sizes, times, marker='s')
    plt.xlabel("Nombre de bits de $n$")
    plt.ylabel("Temps d'exécution de l'attaque (en $s$)")
    plt.title("Temps d'exécution de Pollard rho en fonction du nombre de bits de
$n$")
    plt.show()
```

Tracé du taux de réussite

```
def rate_pollard(min_bits, max_bits, nb_iter):
    sizes = list(range(min_bits, max_bits + 1))
    rates = []

    for bits in sizes:
        success = 0
        for _ in range(nb_iter):
            msg_length = randint(1, 26)
            msg = list(mot_random.generate_random_word(msg_length))
            if est_decrypte(msg, bits):
                success += 1
        rates.append(success/nb_iter)

    return sizes, rates


def trace_rate(min_bits, max_bits, nb_iter):
    sizes, rates = rate_pollard(min_bits, max_bits, nb_iter)

    plt.figure(figsize=(10, 6))
    plt.plot(sizes, rates)
    plt.xlabel("Nombre de bits de n")
    plt.ylabel("Taux de réussite de l'attaque")
    plt.title("Taux de réussite de l'attaque Pollard rho en fonction du nombre de bits de $n$")
    plt.ylim(0, 1.05)
    plt.show()
```